Which Wrong Pages Should an Event Linker Train On?

Negative sampling for the retrieval stage of Event Linking

Rodrigo Quezada
CS 2242, Fall 2026

First draft, November 6, 2026

Abstract

Event linking connects a mention of an event in text to the Wikipedia page about that event. The retrieval stage of the EVELINK system is trained with a contrastive loss against wrong pages, called negatives. This paper asks which wrong pages the retriever should train on. I compare four samplers at a fixed budget: the original in-batch negatives, uniform negatives from the top 1,000 retrieved pages, the seven highest-scoring pages, and ReSa2, a two-stage sampler that prefers pages that score close to the correct page and are similar to it. On 16,692 validation mentions ranked against 5.87 million pages, ReSa2 gives recall at ten of 93.3% against 92.2% for uniform sampling, 87.2% for top-score sampling, and 69.6% for in-batch sampling, with the correct page first 69.5% of the time against 64.3% for uniform. I also measure the inclusion probability, coverage, and variation between runs of each sampler, and I report a training collapse that appears with hard negatives at a higher learning rate and does not recover.

1 Introduction

"He won the Rising Star award in 1996, in his first season with Adelaide."

If I want to link that sentence to Wikipedia, the page I want is "1996 AFL Rising Star." A page about the 2004 Indian Ocean earthquake is easy to rule out. The page for the 1998 AFL Rising Star is harder. It is about the same award. The year is what tells me it is a different event.

That is the problem I study in this paper. When I train an event linker, which wrong pages should I show it?

1.1 Where the link can go wrong

Yu et al. introduced event linking in 2023 [1]. You give the model a mention of an event and the words around it. The model has to find the Wikipedia page that describes that event. We can train it on links that Wikipedia editors already made.

EVELINK does this in two stages. First, a retriever searches for candidate pages. In the original experiments, it retrieves 100. Then a reranker reads the mention and each candidate together and

decides which page belongs at the top. If the correct page never makes the list, the reranker has no way to recover it. I work on that first stage.

The Event Linking paper identifies two types of errors that matter here. A hierarchy error confuses an event with a parent event or a sub-event. A time error selects a related event from the wrong year. In both cases, the wrong page can share much of its wording with the correct page.

1.2 Questions

1. How does ReSa2 [2] change the inclusion probability of each candidate, the coverage of the candidate pool, and the variation between runs?
2. With a fixed compute budget, does ReSa2 put the correct page higher (recall at ten) than the original sampling?

2 What the model learns from a wrong page

The retriever uses two BERT encoders. One reads the mention in context. The other reads the Wikipedia page. Each produces a vector with 1,024 numbers. I take the dot product of the two vectors to get a score:

$$s(m, e) = \langle f_\theta(m),\, g(e) \rangle. \tag{1}$$

Since the page encoder works independently, I can compute the page vectors $g(e)$ once and reuse them for every mention.

During training, I score the correct page and seven wrong pages. I call those seven pages *negatives*. For one mention the loss is

$$L = -\log \frac{e^{s^+}}{e^{s^+} + \sum_{i=1}^{7} e^{s_i}}, \tag{2}$$

where s^+ is the score of the correct page and s_i the score of negative i. The loss goes up when a negative scores close to the correct page or above it.

Suppose the right page scores 10. If the only negative scores -10, the loss is almost zero. The model already knows how to separate those two pages. If the negative scores 9.5, the loss is about 0.47. There is much more for the model to learn from that comparison.

The math puts a bound on this. If a negative scores g points below the correct page, adding it increases the loss by at most e^{-g}:

$$\log\left(1 + \sum_i e^{s_i - s^+}\right) \le \sum_i e^{s_i - s^+} = \sum_i e^{-g_i}. \tag{3}$$

And if I ask how much the loss changes when I raise a negative's score, the answer is that negative's

share of the softmax:

$$\frac{\partial L}{\partial s_i} = p_i = \frac{e^{s_i}}{e^{s^+} + \sum_j e^{s_j}}. \tag{4}$$

In other words, each negative gets a share of the training signal, and its score helps determine how big that share is.

3 How I chose the seven pages

I compared four ways of picking negatives. The first uses the correct pages for the other seven mentions in the same small batch. That is the in-batch baseline. For the other three, I started with the pages that the released retriever ranked highest for that mention. I built a pool of up to 1,000 candidates and excluded the correct page from sampling. Every sampler sees the same input: the pool scores s_i, the score s^+ of the correct page, and the similarity σ_i of each pool page to the correct page (the dot product of the two released page vectors).

Uniform sampling gives every page in the pool the same chance, $k/N = 7/1{,}000$. Top-score sampling always takes the seven highest-scoring pages, so its inclusion probability is 1 for those seven pages and 0 for the rest. ReSa2 makes the choice in two stages.

First, ReSa2 draws 500 pages without replacement. It gives the most weight to pages that score close to the correct page:

$$w_i = \exp\!\big(-\tau\,(s_i - s^+)^2\big), \qquad \tau = 0.25. \tag{5}$$

It draws one page with probability $w_i / \sum_j w_j$, removes it, and draws again from the rest (successive sampling). The weight drops as the score moves away in either direction. Then it compares the 500 survivors to the correct page itself. It keeps the 75 pages whose vectors are most similar to the correct page vector and draws seven of those uniformly.

That last draw gives me a simple bound. Once a page makes the final 75, it has a 7-in-75 chance of being used. So even if a page labeled as wrong is actually a valid link, I can bound how often ReSa2 trains against it and, with equation 4, bound its expected effect on the loss gradient:

$$\mathbb{E}\left[\frac{\partial L}{\partial s_i}\right] \le P(\text{ReSa2 picks } i) \cdot \frac{e^{s_i}}{e^{s^+} + e^{s_i}} \le \frac{7}{75} \cdot \frac{e^{s_i}}{e^{s^+} + e^{s_i}}. \tag{6}$$

4 Experiment setup

I started each run from the released EVELINK model. I froze the page encoder and trained the mention encoder for 1,500 updates with a batch of 32 mentions (four micro-batches of 8), AdamW, a learning rate of 10^{-5} with 10% warm-up, gradient clipping at 1.0, and seven negatives per mention. Because the page encoder is frozen, the pool scores and similarities are fixed before training. I ran each of the four samplers with three seeds, twelve runs in total. Each run took about 56 minutes on one NVIDIA A10G.

Table 1: Validation recall after 1,500 updates, mean ± standard deviation over three seeds. 16,692 mentions ranked against 5.87 million pages.

Sampler	Top-1	Recall@10	Recall@100
Released model, no fine-tuning	42.6	77.7	93.1
In-batch (EVELINK)	33.6 ± 1.9	69.6 ± 1.8	89.8 ± 1.3
Uniform from the pool	64.3 ± 0.2	92.2 ± 0.1	98.4 ± 0.1
Top-score	65.3 ± 1.2	87.2 ± 0.9	95.9 ± 0.6
ReSa2	**69.5 ± 0.3**	**93.3 ± 0.1**	**98.5 ± 0.1**

Then I ranked all 5.87 million pages for each of the 16,692 validation mentions. The metric I focus on is recall at ten: if the correct page appears anywhere in the first ten results, I count that mention as a hit,

$$\text{Recall@}K = \frac{1}{|V|} \sum_{m \in V} \mathbf{1}[\text{rank}(e_m^*) \le K] \,. \tag{7}$$

I also report top-1 and recall at 100.

Before the experiments I reproduced the released pipeline: the released retriever and reranker give 82.88% accuracy on the Wikipedia test set (paper: 82.87%) and 55.33% on the New York Times set (paper: 55.33%). The test set is untouched by everything below.

5 What changed in the rankings

Table 1 gives the results. The released retriever got 77.7 percent recall at ten. After fine-tuning, in-batch sampling got 69.6. Uniform sampling from the pool got 92.2. Taking the seven highest-scoring negatives got 87.2. ReSa2 got 93.3.

Most of the improvement came from drawing negatives from the retrieved pool. ReSa2 added 1.1 percentage points over uniform sampling at ten. The difference was larger at rank one. With uniform sampling, the correct page came first 64.3 percent of the time. With ReSa2, it came first 69.5 percent of the time. The spread between seeds is 0.1 to 0.3 points for both samplers, so the gap is about ten times the noise.

In-batch sampling finished below the released model. At this budget that is expected: seven random negatives per step give almost no gradient (section 2), while the original training used many more negatives per step and ran for longer.

6 Which pages the samplers pick

I also wanted to see which pages the samplers actually picked. I drew negatives 200 times each for 500 validation mentions. Table 2 gives the result.

ReSa2 picked about six of its seven negatives from the 75 pages most similar to the correct page. For uniform sampling, the average was about half a page. Top-score gave me the same seven

Table 2: What each sampler selects: 500 validation mentions, 200 draws per mention, seven negatives per draw. The first column splits the seven negatives by the rank of their similarity to the correct page.

Sampler	Negatives by similarity rank top 25 / 25–75 / 75–300 / rest	Distinct pages (of 1,000)	Overlap of two draws	Score gap to correct page
Uniform	0.17 / 0.34 / 1.55 / 4.95	756	0.00	−13.3
Top-score	4.13 / 1.07 / 1.17 / 0.62	7	1.00	−1.4
ReSa2	2.15 / 3.98 / 0.87 / 0.00	84	0.05	−8.6

pages every time. Two ReSa2 draws shared very few pages (Jaccard overlap 0.05), even though both tended to draw from the same part of the pool. Over 200 draws uniform sampling touched 756 pool pages, ReSa2 84, top-score 7.

The score gap says how hard the negatives are. ReSa2's negatives score 8.6 points below the correct page on average, between uniform (−13.3, easy) and top-score (−1.4, hardest). With raw scores of about 550 and differences of tens of points, the weight in equation 5 is near zero outside $|s_i - s^+| < 4$, so in practice stage 1 is a window around s^+, and stage 2 picks the most similar pages inside that window.

7 What happened at the higher learning rate

At a learning rate of 3×10^{-5}, one ReSa2 run and one top-score run collapsed. The training loss jumped once, near the end of the warm-up, from about 0.13 to about 0.3, and stayed there. When I looked at the scores, I found that the mention encoder gave almost every page a score of about two. It gave roughly the same score to the correct page and to a random page; a healthy model gives about 550 to the correct page and 545 to a random one. The weights stayed finite. The loss over the eight training pages could still look modest while the ranking across millions of pages had fallen apart.

One possible explanation is in the gradient. The gradient of the loss on the mention vector is

$$\nabla_{f_\theta(m)} L = \sum_{i=1}^{7} p_i \left(g(e_i) - g(e^+) \right), \tag{8}$$

with p_i from equation 4. When a negative scores close to the correct page, it gets a substantial share of the training signal. If the two page vectors are also similar, the update depends on the small difference between them. My hypothesis is that repeated updates at the higher learning rate weakened the part of the mention vector that lined up with the page vectors. The scores are consistent with that explanation. Uniform and in-batch runs, whose negatives are far from the correct page, never jumped.

I tried training the collapsed model for another 1,500 updates. I tried ReSa2 negatives in one run and uniform negatives in another. Recall at ten stayed at zero in both. When I restarted from

the released checkpoint and used a learning rate of 10^{-5}, all twelve runs trained without collapsing, and every sampler scored better than at the higher rate.

8 Conclusion

I started with the difference between the 1996 award and the 1998 award. The results make that example more specific for me. I want the retriever to see pages that are close enough to make it pay attention to the year. I also want it to see a different set of pages the next time it trains on that mention. In these runs, ReSa2 gave me that variety and put the correct page first more often than any of the other samplers I tested.

References

[1] Xiaodong Yu, Wenpeng Yin, Nitish Gupta, and Dan Roth. Event Linking: Grounding Event Mentions to Wikipedia. In *Proceedings of EACL*, 2023.

[2] Li et al. ReSa2: Re-sampling for Retrieval Training. In *Proceedings of ACML*, 2025.

[3] Ledell Wu, Fabio Petroni, Martin Josifoski, Sebastian Riedel, and Luke Zettlemoyer. Scalable Zero-shot Entity Linking with Dense Entity Retrieval. In *Proceedings of EMNLP*, 2020.